Class Object OOP Python

Explain how to create class and object in python and oops concept with implementation in python

Great 👍 Let's understand **Class, Object, and OOP concepts in Python** in a clear and practical way.

🔷 1. What is a Class?

- A **class** is a blueprint or template.
- It defines:
  - **Attributes (variables)** → data
  - **Methods (functions)** → behavior
- It does not consume memory until an object is created.

✅ Syntax:

```python
class ClassName:
    # attributes
    # methods
```

🔷 2. What is an Object?

- An **object** is an instance of a class.
- It represents a real-world entity.
- Memory is allocated when an object is created.

✅ Creating Object:

```python
object_name = ClassName()
```

🔷 3. Example: Creating Class and Object

```python
class Student:
    def __init__(self, name, age):
        self.name = name      # attribute
        self.age = age

    def display(self):       # method
        print("Name:", self.name)
        print("Age:", self.age)

# Creating object
s1 = Student("Chetan", 22)

# Accessing method
s1.display()
```

🔹 Output:

```
Name: Chetan
Age: 22
```

🔷 4. Understanding `__init__()` Constructor

- `__init__()` is a **constructor**
- Automatically called when object is created
- Used to initialize data members

```python
def __init__(self, parameter):
```

🔹 What is `self`?

- `self` refers to the current object.
- It is used to access class variables and methods.

🟢 OOPS Concepts in Python

Python supports 4 main OOP principles:

1. Encapsulation
2. Abstraction

3. Inheritance
4. Polymorphism

🔶 1. Encapsulation

- Wrapping data and methods together inside a class.
- Restricting direct access to variables.
- Use `_` or `__` to make private variables.

✅ Example:

```python
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance   # private variable

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
account.deposit(500)

print(account.get_balance())
```

🔹 Output:

```code
1500
```

❌ `account.__balance` → Error (cannot access directly)

🔶 2. Abstraction

- Hiding implementation details.
- Showing only essential features.
- Achieved using `ABC` (Abstract Base Class).

✅ Example:

```python
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius * self.radius

c = Circle(5)
print(c.area())
```

🔶 3. Inheritance

- One class inherits properties from another.
- Promotes code reusability.

✅ Example:

```python
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

d = Dog()
d.speak()
d.bark()
```

🔹 Output:

```
Animal speaks
Dog barks
```

🔶 Types of Inheritance in Python

1. Single
2. Multiple
3. Multilevel
4. Hierarchical
5. Hybrid

✅ Multiple Inheritance Example:

```python
class Father:
    def skill1(self):
        print("Gardening")

class Mother:
    def skill2(self):
        print("Cooking")

class Child(Father, Mother):
    pass

c = Child()
c.skill1()
c.skill2()
```

🔶 4. Polymorphism

- Same method name, different behavior.

✅ Method Overriding Example:

```python
class Bird:
    def fly(self):
        print("Some birds can fly")

class Sparrow(Bird):
    def fly(self):
        print("Sparrow flies high")

b = Sparrow()
b.fly()
```

🔹 Output:

```
Code
Sparrow flies high
```

🔷 Important OOP Features in Python

🔹 1. Class Variables vs Instance Variables

```python
class Car:
    wheels = 4  # class variable

    def __init__(self, brand):
        self.brand = brand  # instance variable

c1 = Car("BMW")
c2 = Car("Audi")

print(c1.wheels)   # 4
print(c2.brand)    # Audi
```

🔹 2. Static Method

```python
class Math:

    @staticmethod
    def add(a, b):
        return a + b

print(Math.add(5, 3))
```

🔹 3. Class Method

```python
class Employee:
    company = "Google"

    @classmethod
```

```
    def change_company(cls, name):
        cls.company = name

Employee.change_company("Microsoft")
print(Employee.company)
```

🔥 Real World Analogy

Real World	Programming
Car Design	Class
Actual Car	Object
Driving	Method
Color, Speed	Attributes

🧠 Summary

- Class → Blueprint
- Object → Instance
- `__init__()` → Constructor
- `self` → Current object
- Encapsulation → Data hiding
- Abstraction → Hiding implementation
- Inheritance → Reusing code
- Polymorphism → Same method, different behavior

If you want, I can also give:

- 🔹 OOP interview questions
- 🔹 Difference between Python and Java OOP
- 🔹 Advanced concepts (Magic methods, Dunder methods, MRO)
- 🔹 Practice problems for OOP

Just tell me 👍

```
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius * self.radius

c = Circle(5)
print(c.area())
```

Explain this code and how abc is import and why

Good question 👍 This is an important OOP concept — **Abstraction using ABC in Python**.

Let's break your code step by step.

🔷 1️⃣ What is `abc` in Python?

`abc` stands for:

> **A**bstract **B**ase **C**lass

It is a built-in Python module used to create:

- Abstract classes
- Abstract methods

We import it like this:

```python
from abc import ABC, abstractmethod
```

🔹 Why do we import this?

Because Python does NOT have direct abstract class support like Java.

So we use:

- `ABC` → To make a class abstract
- `@abstractmethod` → To declare abstract methods

🔷 2️⃣ Understanding Your Code Step-by-Step

✅ Step 1: Importing ABC

```python
from abc import ABC, abstractmethod
```

- `ABC` → Base class to create abstract class
- `abstractmethod` → Decorator to define abstract methods

✅ Step 2: Creating Abstract Class

```python
class Shape(ABC):
```

Here:

- `Shape` is an abstract class.
- It inherits from `ABC`.
- This tells Python: "This class can contain abstract methods."

✅ Step 3: Creating Abstract Method

```python
@abstractmethod
def area(self):
    pass
```

What is happening here?

- `@abstractmethod` → Forces child classes to implement this method.
- `area()` has no body.
- It just defines a rule.

This means:

> ❌ You CANNOT create object of Shape directly
> ✅ Any subclass MUST implement `area()`

❌ Try This:

```python
s = Shape()
```

It will give error:

```
TypeError: Can't instantiate abstract class Shape
```

Because abstract class cannot create object.

🔷 4️⃣ Creating Child Class

```python
class Circle(Shape):
```

Now `Circle` inherits from `Shape`.

Since `Shape` has an abstract method `area()`,
`Circle` MUST implement it.

✅ Constructor

```python
def __init__(self, radius):
```

```
    self.radius = radius
```

Stores radius in object.

✅ Implementing Abstract Method

```python
def area(self):
    return 3.14 * self.radius * self.radius
```

Now abstract method is implemented.

So Circle becomes a **concrete class**.

🔷 5️⃣ Creating Object

```python
c = Circle(5)
print(c.area())
```

Output:

```
78.5
```

🔷 Why Use Abstract Class?

Abstraction means:

> Hide implementation details and enforce structure.

Real-world Example:

Imagine:

- Shape → common structure
- Circle → specific implementation
- Rectangle → different implementation

Example:

```python
class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width
```

Now both classes follow same structure.

🔷 Why Is ABC Important?

Without `abc`:

- No guarantee that subclass implements required method
- Code structure becomes weak
- No enforcement

With `abc`:

- Enforces method implementation
- Ensures consistency
- Helps in large projects
- Improves code design

🔷 Concept Summary

Concept	Meaning
ABC	Abstract Base Class
@abstractmethod	Forces child to implement method
Abstract class	Cannot create object
Concrete class	Can create object

🔥 Real-Life Analogy

Think of:

Shape = Rules

- Every shape must have area()

But shape itself doesn't know how to calculate it.

Circle and Rectangle know how.

🔷 Final Concept Understanding

Flow of execution:

1. Import abc
2. Create abstract class Shape
3. Define abstract method area()
4. Create Circle class
5. Implement area()
6. Create object
7. Call method

If you want, I can also explain:

- 🔹 Difference between abstraction & encapsulation
- 🔹 How Python internally checks abstract method
- 🔹 Multiple abstract methods example
- 🔹 Real interview explanation answer format

Tell me 👍

```
class Father:
    def skill1(self):
        print("Gardening")

class Mother:
    def skill2(self):
```

```
        print("Cooking")

class Child(Father, Mother):
    pass

c = Child()
c.skill1()
c.skill2()
```

If we have same method doens't it will give same error like java

Excellent question 👏 This shows you're thinking in Java terms — good!

Short answer:

> ❌ No, Python does NOT give a compile-time error like Java.
> ✅ Python uses something called **MRO (Method Resolution Order)** to decide which method to call.

Let’s understand properly.

🔷 In Java (Why Error Happens)

In Java:

- Multiple inheritance of classes is NOT allowed.
- Because of **Diamond Problem** ambiguity.
- Java allows multiple inheritance only through interfaces.

So Java prevents ambiguity at compile time.

🔷 In Python

Python **allows multiple inheritance**.

If two parent classes have the same method name, Python:

> Uses **MRO (Method Resolution Order)** to decide which one to call.

🔷 Let's Modify Your Example

```python
class Father:
    def skill(self):
        print("Gardening")

class Mother:
    def skill(self):
        print("Cooking")

class Child(Father, Mother):
    pass

c = Child()
c.skill()
```

🔹 Output:

```
Gardening
```

Why?

Because Python checks classes in this order:

```
Child → Father → Mother → object
```

It finds `skill()` in `Father` first.

🔷 What is MRO?

MRO = Method Resolution Order

It defines the order in which Python searches for methods.

You can check it:

```python
print(Child.mro())
```

Output:

```
Code
[<class '__main__.Child'>,
 <class '__main__.Father'>,
 <class '__main__.Mother'>,
 <class 'object'>]
```

So Python checks:

1. Child
2. Father
3. Mother
4. object

🔷 What If You Change Order?

```python
class Child(Mother, Father):
    pass
```

Now:

```python
c = Child()
c.skill()
```

Output:

```
Code
Cooking
```

Because now order is:

```
Code
Child → Mother → Father → object
```

🔥 So Why No Error in Python?

Because:

- Python resolves ambiguity dynamically.
- It follows C3 Linearization algorithm.
- It avoids diamond problem confusion using MRO.

Java avoids complexity by banning multiple inheritance of classes.

Python handles it smartly.

🔷 Diamond Problem Example

```python
class A:
    def show(self):
        print("A")

class B(A):
    def show(self):
        print("B")

class C(A):
    def show(self):
        print("C")

class D(B, C):
    pass

d = D()
d.show()
```

Output:

```
B
```

Why?

MRO:

```
D → B → C → A → object
```

So it picks B's method.

🔷 Important Interview Point

In Python:

- No compile-time error
- Uses MRO
- Order of inheritance matters
- Leftmost parent gets priority

🔥 Final Comparison

Feature	Java	Python
Multiple Inheritance	❌ Not allowed	✅ Allowed
Same method conflict	Compile error	Uses MRO
Resolution time	Compile time	Runtime

Since you're preparing for interviews, remember this line:

> "Python resolves multiple inheritance conflicts using C3 Linearization and Method Resolution Order (MRO)."

If you want, I can now explain:

- 🔹 C3 Linearization in simple way
- 🔹 How `super()` works in multiple inheritance
- 🔹 When multiple inheritance becomes dangerous
- 🔹 Real industry best practice

Tell me 👍

What is pass

Good 👍 simple but very important concept.

🔷 What is `pass` in Python?

`pass` is a **null statement**.

It means:

> "Do nothing."

Python requires some code inside blocks like:

- functions
- classes
- loops
- if statements

If you don't want to write anything yet, you use `pass`.

🔷 Why Do We Need It?

Python does not allow empty blocks.

❌ This gives error:

```python
if True:
```

Error:

```code
IndentationError: expected an indented block
```

So we use `pass`.

🔷 Example 1: In Function

```python
def my_function():
    pass
```

This function does nothing but is syntactically valid.

🔷 Example 2: In Class

```python
class MyClass:
    pass
```

Creates an empty class.

🔷 Example 3: In Loop

```python
for i in range(5):
    pass
```

Loop runs but does nothing.

🔷 Example 4: In If Statement

```python
x = 10

if x > 5:
    pass
else:
    print("Small number")
```

If condition is true, nothing happens.

🔷 Difference Between `pass`, `continue`, and `break`

Keyword	Meaning
pass	Do nothing
continue	Skip current loop iteration

Keyword	Meaning
break	Exit loop completely

Example:

```python
for i in range(5):
    if i == 2:
        pass
    print(i)
```

Output:

```code
0
1
2
3
4
```

Because `pass` does nothing.

🔷 Important Use Case in OOP

In your earlier example:

```python
class Child(Father, Mother):
    pass
```

Why `pass`?

Because:

- You don't want to add anything new.
- Child simply inherits everything.
- But Python still needs a block.

So `pass` is used.

🔥 Interview Definition

> `pass` is a placeholder statement used when a statement is syntactically required but no action is needed.

If you want, I can also explain:

- 🔹 Difference between `pass` and `...` (ellipsis)
- 🔹 When to use `NotImplemented`
- 🔹 Common beginner mistakes with pass

Tell me 👍

```
class Bird:
    def fly(self):
        print("Some birds can fly")

class Sparrow(Bird):
    def fly(self):
        print("Sparrow flies high")

b = Sparrow()
b.fly()
```

How will I access bird class method then

Very good question 👏
You are now thinking about **method overriding and parent method access** — important OOP concept.

In your code:

Python

```python
class Bird:
    def fly(self):
        print("Some birds can fly")

class Sparrow(Bird):
    def fly(self):
        print("Sparrow flies high")
```

```
b = Sparrow()
b.fly()
```

Output:

```
Code
Sparrow flies high
```

Because:

👉 `Sparrow` overrides the `fly()` method of `Bird`.

So when you call:

```python
b.fly()
```

Python first checks in `Sparrow`, finds `fly()`, and stops there.

🔷 How to Access Parent (Bird) Method?

You use:

✅ 1️⃣ Using `super()`

```python
class Sparrow(Bird):
    def fly(self):
        super().fly()   # calls Bird's fly()
        print("Sparrow flies high")
```

Now:

```python
b = Sparrow()
b.fly()
```

Output:

```
Code
```

```
Some birds can fly
Sparrow flies high
```

🔹 What is `super()`?

`super()` gives access to the **parent class methods**.

It means:

> "Go to my parent class and call its method."

✅ 2️⃣ Directly Calling Parent Class (Less Recommended)

```python
class Sparrow(Bird):
    def fly(self):
        Bird.fly(self)
        print("Sparrow flies high")
```

This also works.

But ❗ best practice is to use `super()`.

🔷 Why Use `super()` Instead of Bird.fly(self)?

Because:

- It works properly in **multiple inheritance**
- It respects MRO (Method Resolution Order)
- Cleaner and safer

🔷 Important Concept: Method Overriding

When child defines same method:

```python
def fly(self):
```

It overrides parent method.

Parent method is NOT deleted.

It is just hidden.

You can still access it using `super()`.

🔥 Real Interview Explanation

> In Python, when a child class overrides a method, we can access the parent class method using `super()`. It follows the Method Resolution Order (MRO).

🔷 Bonus: Check MRO

```python
print(Sparrow.mro())
```

Output:

```
[Sparrow, Bird, object]
```

Python searches in this order.

🧠 Final Understanding

Situation	What Happens
Same method in child	Child version runs
Want parent version	Use `super()`
Direct call	`Parent.method(self)`

If you want, I can next explain:

- 🔹 How `super()` works internally

- 🔹 super() in multiple inheritance
- 🔹 super() in constructor (`__init__`)
- 🔹 Common mistakes in overriding

Tell me 👍